

Batch based architecture

SQL Server operational DB
↓
Nightly SSIS extract
↓
11:00 PM: copy changed data
↓
SQL Server DW

SQL Server

INSERT / UPDATE / DELETE

Datastream CDC

BigQuery

DataStream

"Kill the nightly extracts"

CURRENT

TARGET

SQL Server DW

BigQuery

Database server

No server to manage

SQL Server instance

No equivalent instance to size

Tables

BigQuery tables

T-SQL

✓ GoogleSQL

✓ SQL Agent

✓ Scheduled queries / orchestration

✓ SSIS transformations

✓ Dataform / sometimes Dataflow

Backups/patching

Managed by Google

Hardware capacity

Managed cloud infrastructure ✓

"serverless, no instance, no patching."

Operational systems



SQL Server



Datastream



BigQuery



Bronze → Silver → Gold

Dataform



Gold marts



Power BI

...

Bronze

Applicant_ID	First_Name	Date	Program_Code
38271	JOHN	10/04/2026	A17

Keep the source representation with minimal intervention.

Silver

applicant_id	first_name	application_date	program_id
38271	John	2026-10-04	A17

You've standardized names, dates, data types, duplicates and perhaps joined reference data.

Gold

Program	Month	Applications	Approved	Approval_Rate
A17	2026-10	14,291	11,087	77.6%

Now you've created something intentionally designed for analytics.

Initial migration

SQL Server

BACKFILL

BigQuery

1

After that...

2

INSERT

UPDATE

DELETE

Datastream CDC

BigQuery

continuous change stream

migration / Replication



Migration normally means moving an existing body of data from A to B. Suppose SQL Server already contains 500 million rows. Those existing rows have to get into BigQuery. That's the **backfill**.

A

Step 1 – Backfill

SQL Server

500 million existing rows



Datastream



BigQuery

B

Step 2 – Continuous replication

New INSERT

UPDATE

DELETE



Datastream → BigQuery

SQL ~~server~~ Server

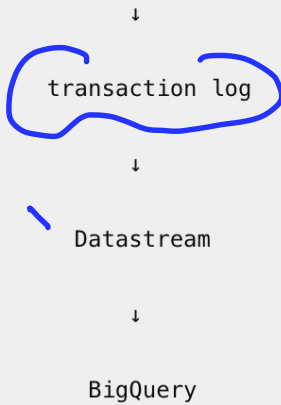
customer_id	name	city
101	Ahmed	Des Moines
102	Sarah	Ames
103	John	Waterloo

INSERT INTO customers ...

UPDATE customers ...

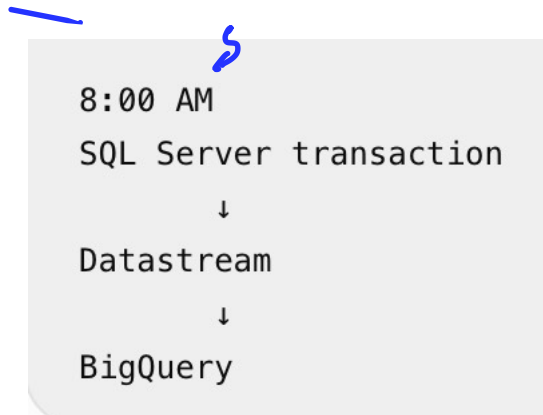
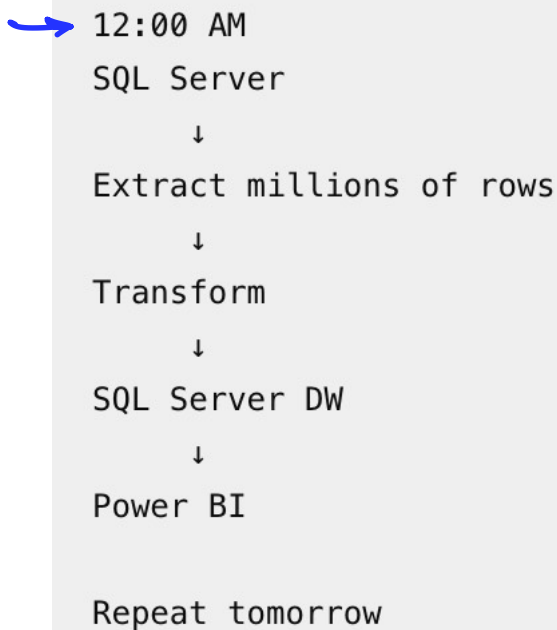
DELETE FROM customers ...

✓ SQL Server transaction
✓ UPDATE customers
SET city = 'Cedar Rapids'
WHERE customer_id = 101;



So the important word is **incremental**.

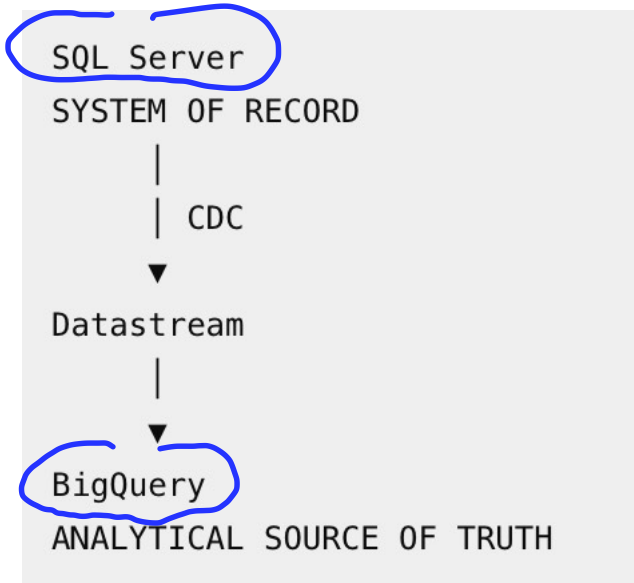
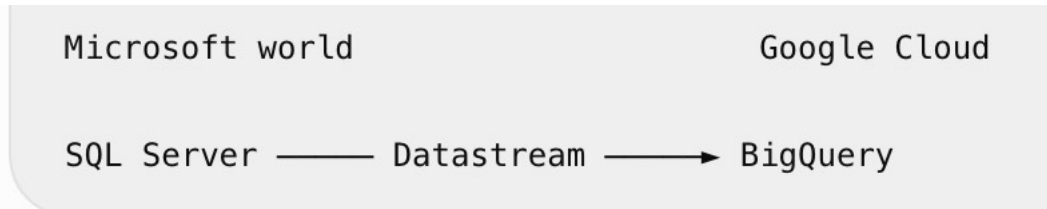
Extract Window



continuous/low-latency replication;

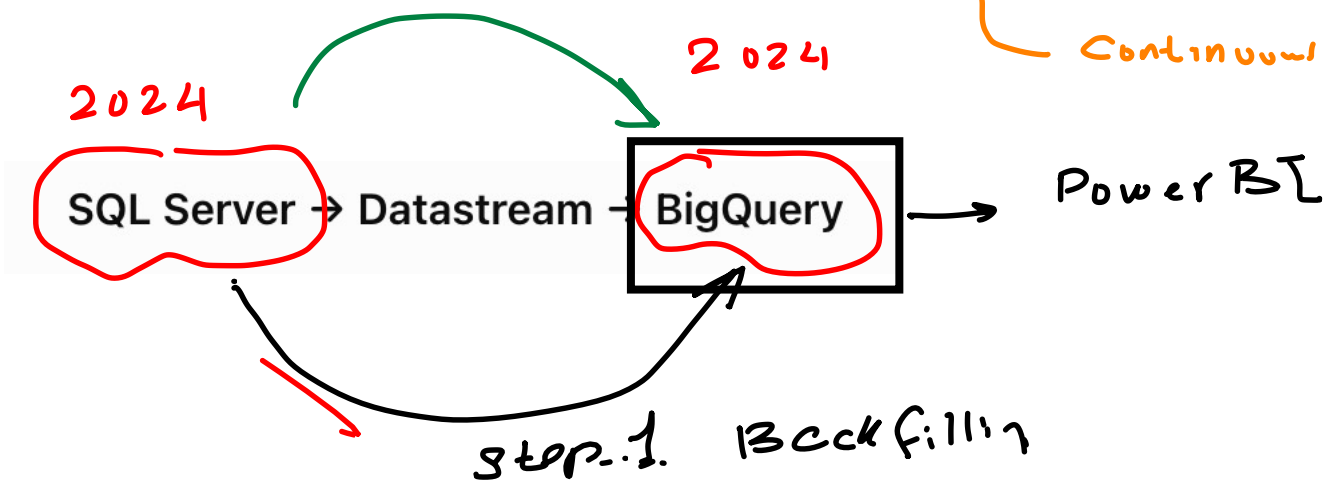
~~instantaneous or zero-latency r~~

"SQL Server is a supported Datastream source — your SQL Server → BigQuery path is first-class."

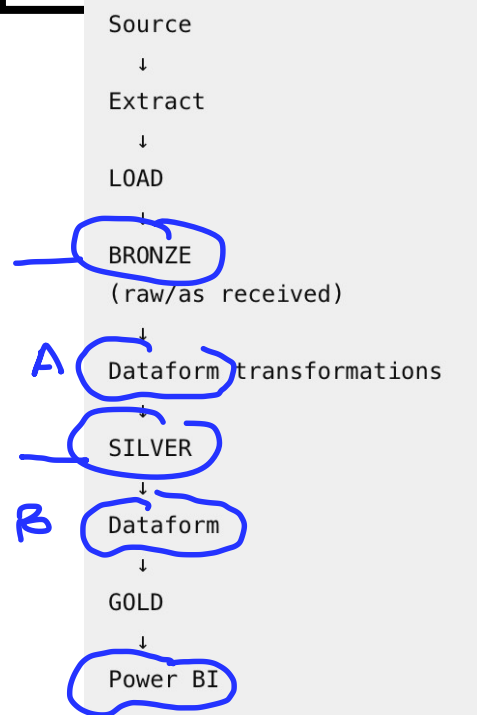
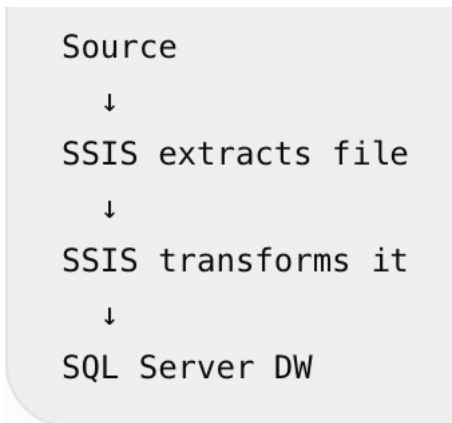
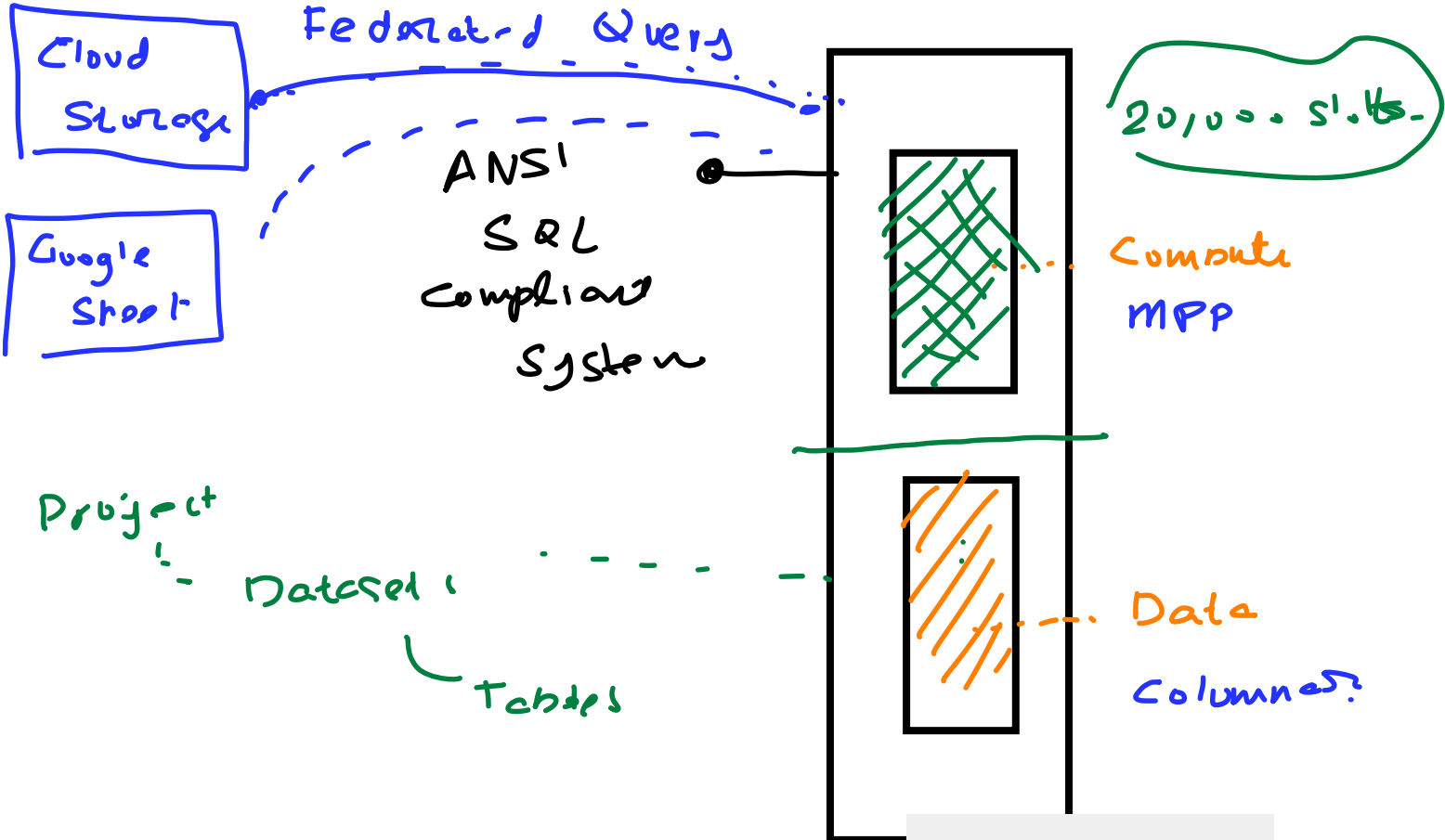


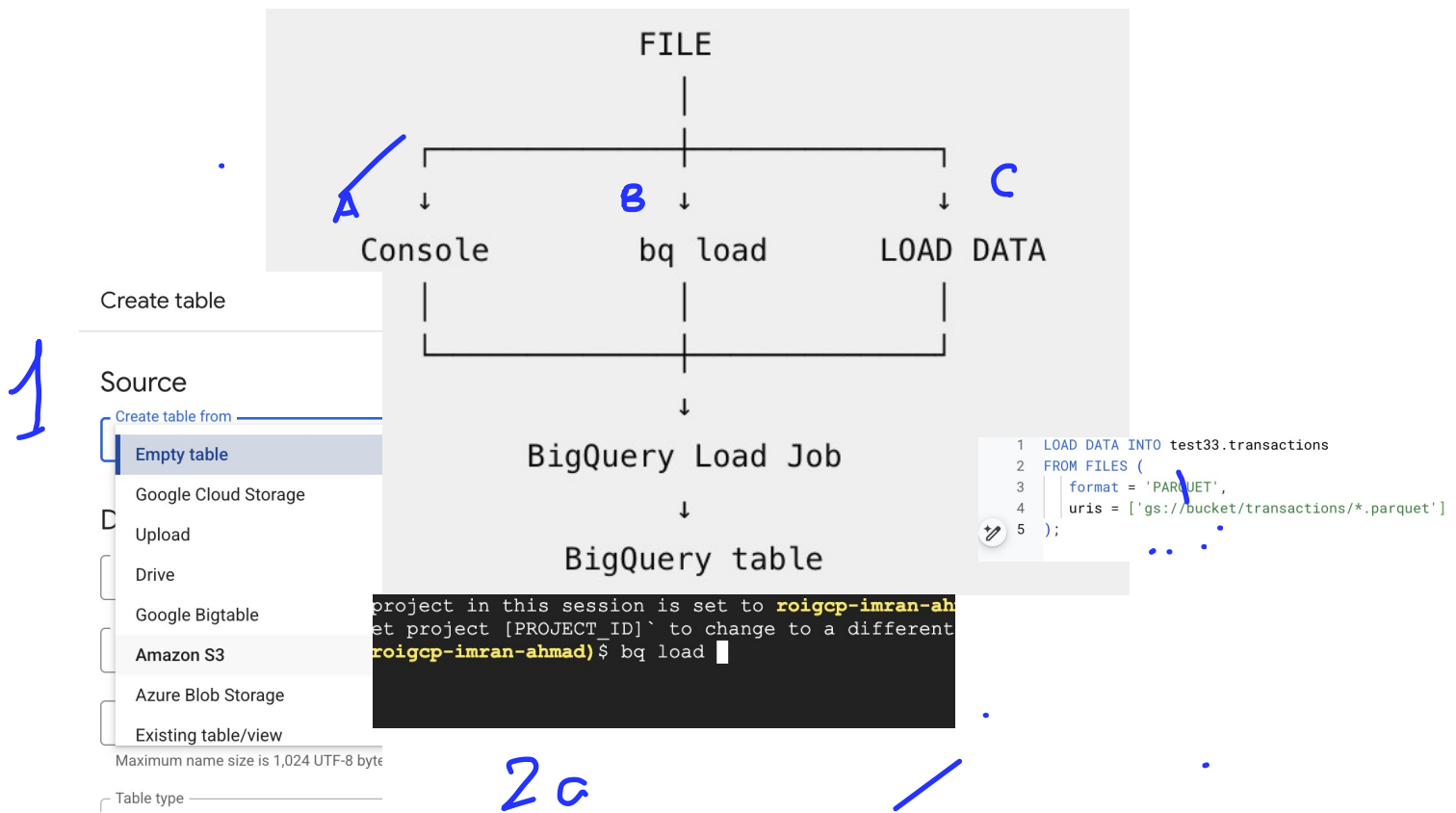
Step 2: Continuous Replication

... near zero latency
Continuous replication

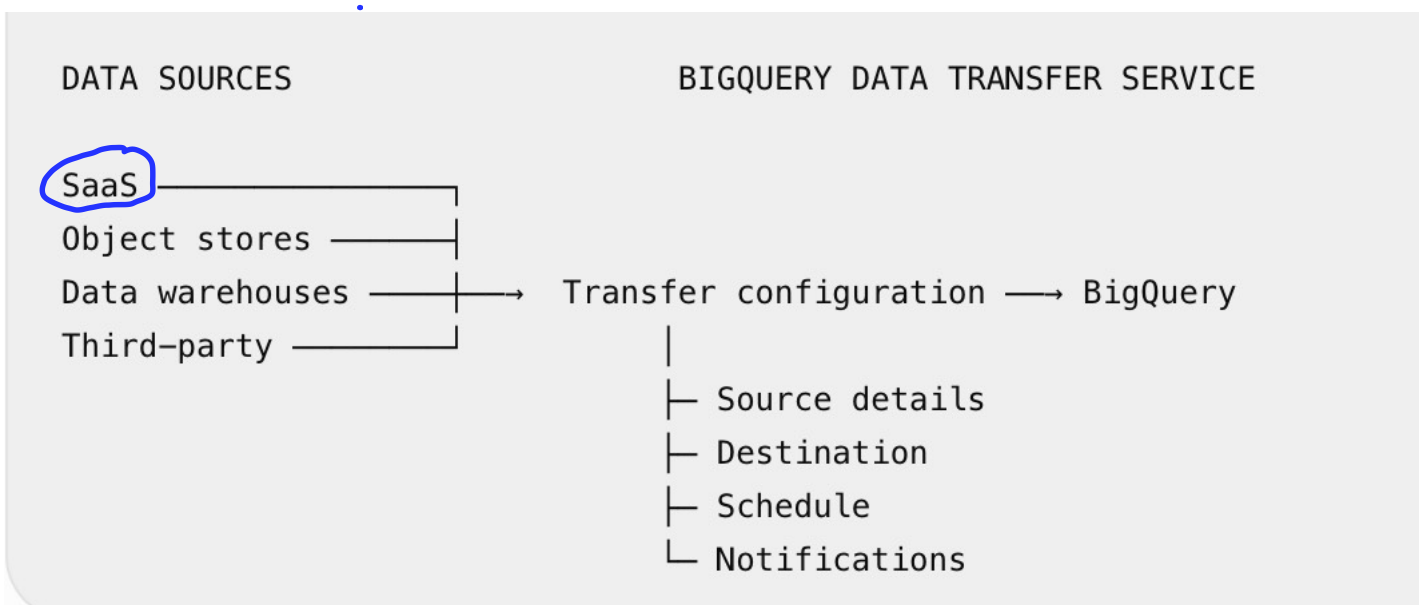


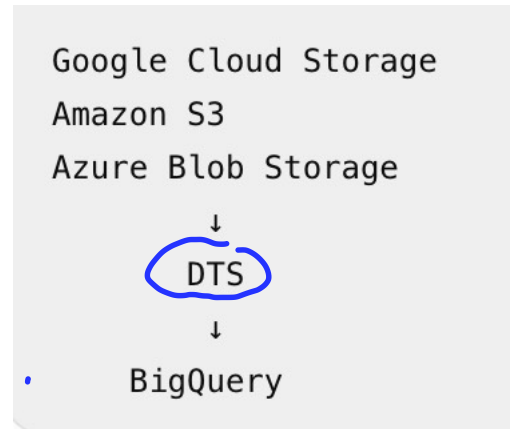
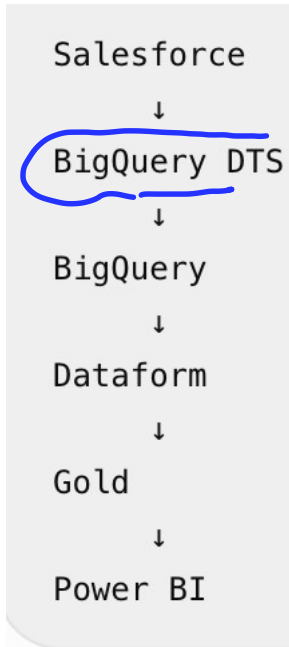
Serverless





Datastream is mainly about continuous CDC replication. BigQuery Data Transfer Service is mainly about managed, scheduled ingestion into BigQuery.





Dataform

No Automated Dependency Management:

True Version Control & Environments:

Built-in Data Quality Testing (Assertions):

For DOM DoIT's target architecture, which service is intended to become the analytical single source of truth?

BigQuery

In the Bronze → Silver → Gold pattern, what is the main purpose of the Bronze layer?

Rose Steffensmeier 4:02 PM

Bring in the data, don't make changes to it

Sampath Paripalli 4:02 PM

source of truth

Ajay Chagamreddy 4:02 PM

Raw data Ingestion

Ajit Chincholikar 4:03 PM

Source data or as close as possible

sowmya ramaiahgari 4:03 PM

retain source data

✓

✓

✓

✓

True or False, Datastream is primarily used to build Silver-to-Gold SQL transformation models.

dataflow

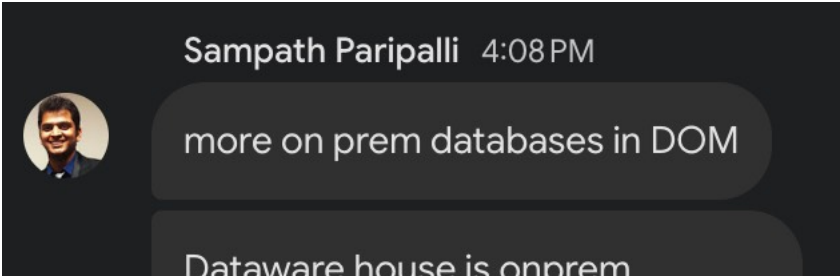
What is the main difference between loading a file into a native BigQuery table and using an external table?

Sampath Paripalli 4:05 PM

 External table query directly the source

A department receives a Parquet file every night and wants a managed recurring process to load it into BigQuery. Which service best fits?

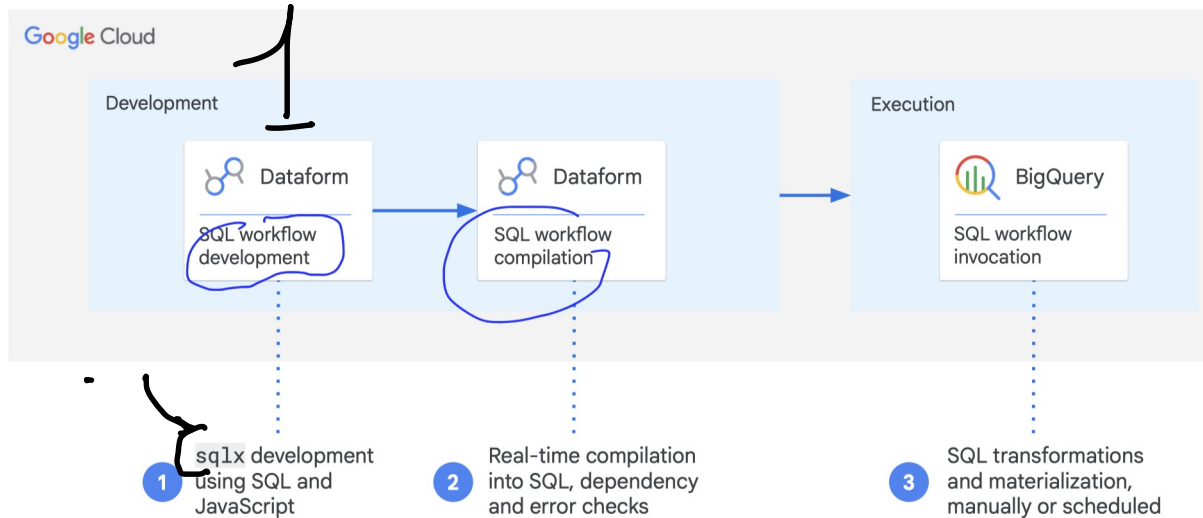
Why is Datastream important for DOM DoIT's SQL Server migration?



Comparison: UDFs vs. Stored Procedures

Feature / Capability	User-Defined Function (UDF) PDF	Stored Procedure PDF + 1
Invocation Syntax	Called inline inside SQL expressions: <code>SELECT my_udf(col) FROM table</code> PDF	Invoked using the <code>CALL</code> statement: <code>CALL my_sproc(arg1, ...);</code> PDF + 1
Purpose	Transformation, calculation, and row cleansing PDF	Workflow execution, administrative tasks, and multi-step orchestration PDF + 2
Data Manipulation (DML/DDL)	Read-only: Cannot run <code>INSERT</code> , <code>UPDATE</code> , <code>DELETE</code> , <code>CREATE</code> , or <code>DROP</code>	Full support: Can create, alter, insert, update, delete, or drop tables PDF + 2
Procedural Logic	Single expression or subquery (or JavaScript logic) PDF	Supports <code>DECLARE</code> , <code>SET</code> , <code>IF</code> , <code>WHILE</code> , and <code>BEGIN...EXCEPTION</code> PDF + 2
Return Type	Returns a scalar value or a table result PDF	Returns execution status, result sets, or updates <code>OUT</code> variables PDF + 1
Languages Supported	GoogleSQL, JavaScript, or Remote (Cloud Run) PDF	GoogleSQL scripting, Apache Spark (Python/Java/Scala) PDF

SQL workflow development and compilation run in Dataform, workflow execution runs in BigQuery



project.dataset.table,

⌘
: \${ref("source_table")}

```
// definitions/declarations raw_case_records.sqlx 1
config {
  type: "declaration",
  database: "iowa-dom-data",
  schema: "bronze_raw",
  name: "raw_case_records",
  description: "Raw case records replicated from SQL Server via Datastream"
}
```

↓

```

-- definitions/silver/stg_case_records.sqlx
config {
  type: "table",
  schema: "silver_conformed",
  description: "Cleaned case records with standard county IDs"
}

SELECT
  case_id,
  ✓ UPPER(TRIM(county_code)) AS county_code,
  ✓ PARSE_DATE('%Y-%m-%d', event_date) AS case_date,
  SAFE_CAST(case_amount AS NUMERIC) AS case_amount,
  ✓ CURRENT_TIMESTAMP() AS processed_at
FROM ${ref("raw_case_records")} -- Upstream dependency declared
WHERE case_id IS NOT NULL

```

```

-- definitions/gold/mart_monthly_county_cases.sqlx
config {
  type: "table",
  schema: "gold_marts",
  description: "Monthly case summaries for Power BI dashboards",
  bigquery: {
    partitionBy: "case_month",
    clusterBy: ["county_code"]
  }
}

SELECT
  DATE_TRUNC(case_date, MONTH) AS case_month,
  county_code,
  COUNT(case_id) AS total_cases,
  SUM(case_amount) AS total_obligated_amount
FROM ${ref("stg_case_records")} -- References the Silver model
GROUP BY 1, 2

```